

Reproducing PINNsFormer

A Transformer-based framework for physics-informed neural networks
Zhao, Ding and Prakash, ICLR 2024

Diell Kryeziu
Advanced Topics in Machine Learning, USI

Outline

1. Background: what is a PDE, what is a PINN, and where PINNs fail
2. The paper's idea: PINNsFormer
3. What I set out to check, and how
4. Results: what reproduced and what did not
5. My own extra experiments
6. Conclusion and demo

Background: what is a PDE?

A PDE is a rule that a function $u(x, t)$ must satisfy everywhere.

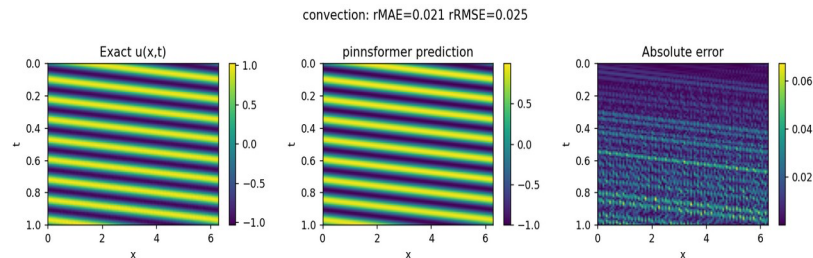
Example, the convection equation:

$$u_t + 50 \cdot u_x = 0$$

u_t = how fast u changes in time, u_x = the slope in space.
Meaning: the shape of u is carried to the right at speed 50.

Plus two extra conditions:

- Initial condition: at $t = 0$, $u = \sin(x)$
- Boundary condition: the left and right edges match (periodic)



Left: the exact solution. x goes left to right, t goes top to bottom. The stripes are the sine wave moving to the right as time passes.

The three equations in the paper

- Convection (transport)
- 1D-reaction (growth toward 1)
- 1D-wave (a vibrating string)

Background: what is a PINN?

A neural network that takes (x, t) and outputs a number $\hat{u}(x, t)$.

There is no training data. Instead, the loss measures how badly the network breaks the rules:

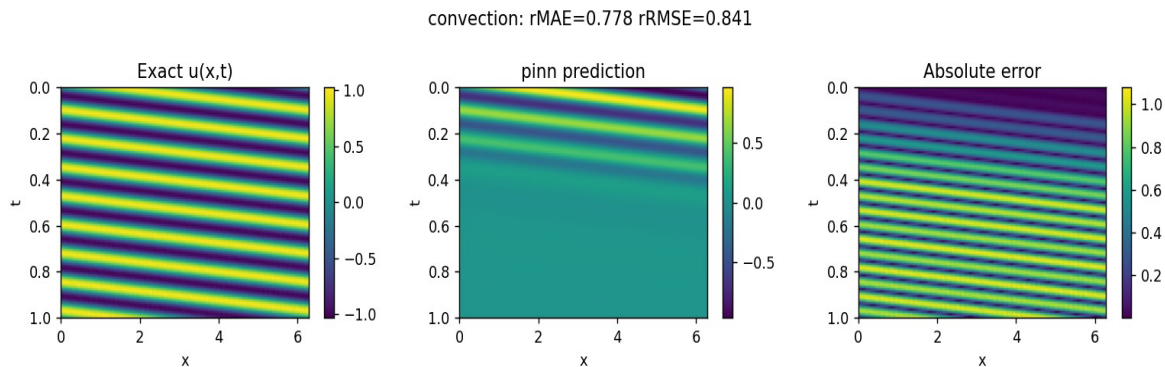
- Residual: plug $\hat{u}$ into the PDE at many grid points. Should be 0.
- Initial condition: $\hat{u}(x, 0)$ should equal $\sin(x)$.
- Boundary: the two edges should match.

Loss = residual + initial + boundary. Train until it is close to 0.

Details that matter later

- Derivatives like u_t and u_x are computed exactly by autograd.
- The grid is 101×101 points (10 201 points).
- The optimiser is L-BFGS: big well-aimed steps, each with a line search of up to 20 loss evaluations.
- The only randomness is the initial weights, set by a seed.
- Error is measured against the exact solution: rMAE = 0 is perfect, about 1 is useless.

Background: sometimes PINNs fail



Two known failure cases

- Convection with speed 50
- 1D-reaction with rate 5

Both have easy exact solutions, and both make plain PINNs fail.

My own plain PINN on convection. Left: exact. Middle: prediction. Right: error. rMAE = 0.78.

The prediction is right near $t = 0$ and then goes flat. Why? A flat function has zero derivatives, so it makes the residual zero for free. Only the initial-condition term complains, and only at $t = 0$. The optimiser takes the cheap way out. This is a known problem (Krishnapriyan et al. 2021), and it is what the paper wants to fix.

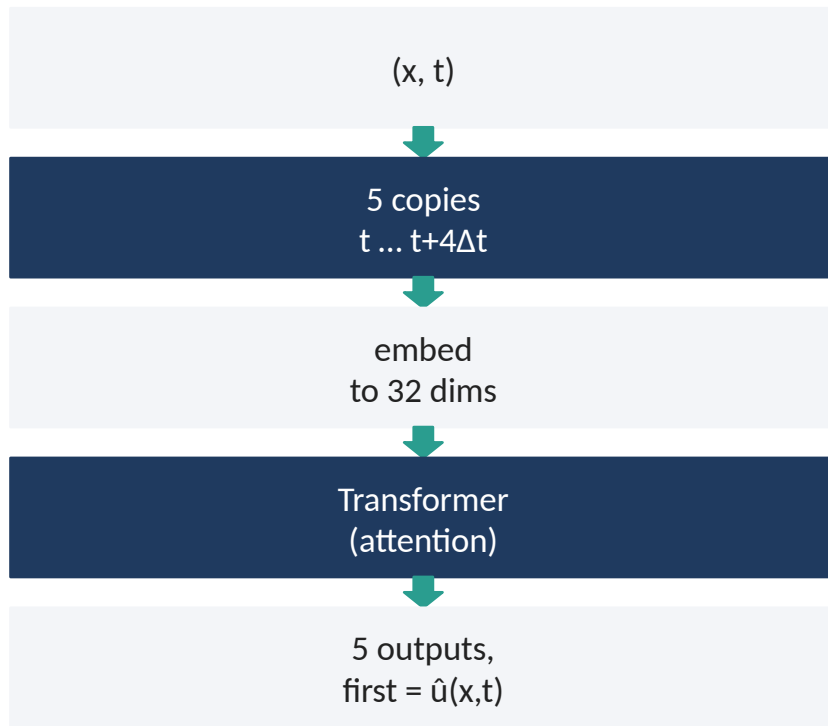
The paper's idea: PINNsFormer

The authors' explanation for the failure:

A PINN looks at each point (x, t) on its own. But in physics, what happens at $t + \Delta t$ depends on what happens at t . A point-wise model cannot see that.

Their fix, in four steps:

- Turn each point into a short sequence of 5 points: $t, t+\Delta t, \dots, t+4\Delta t$.
- Feed the sequence to a small Transformer (encoder + decoder with attention), so the 5 points can 'look at each other'.
- Apply the loss to all 5 outputs, except the initial condition, which only applies to the first (it is the only one at $t = 0$).
- Use a new activation function: $\omega_1 \cdot \sin(x) + \omega_2 \cdot \cos(x)$, called Wavelet.



What I set out to check

I turned the paper into five claims that can be true or false:

1. PINNsFormer fixes the failure on convection and 1D-reaction, while three plain baselines (PINN, QRes, FLS) fail. (their Table 1)
2. The Wavelet activation is necessary: with ReLU, Sigmoid or Sin it fails again. (Table 6)
3. It also works on the wave equation and combines with other training tricks. (Table 2)
4. It costs only about 3× the time and 2× the memory of a PINN. (Table 5)
5. The reason it works is the temporal dependencies captured by the sequence and attention. (their explanation, not stated as a claim)

Not done, for lack of compute: Navier–Stokes, the NTK training trick, the $k/\Delta t$ sensitivity grid.

How I did it

I used the authors' code from GitHub.

- It has one Jupyter notebook per experiment (16 in total).
- I wrote one Python script that does the same as any notebook from the command line, so I could run many variants.
- I checked the script against all 10 relevant notebooks line by line: same grid, same loss terms, same optimiser, same initialisation, same metrics.
- Everything ran on Google Colab's free GPU.

In numbers

- 34 training runs
- about 7 GPU hours
- 3 random seeds instead of 1
- 5 kinds of extra experiments the paper does not have

Each run logs the loss, the error, the time per step and the GPU memory.

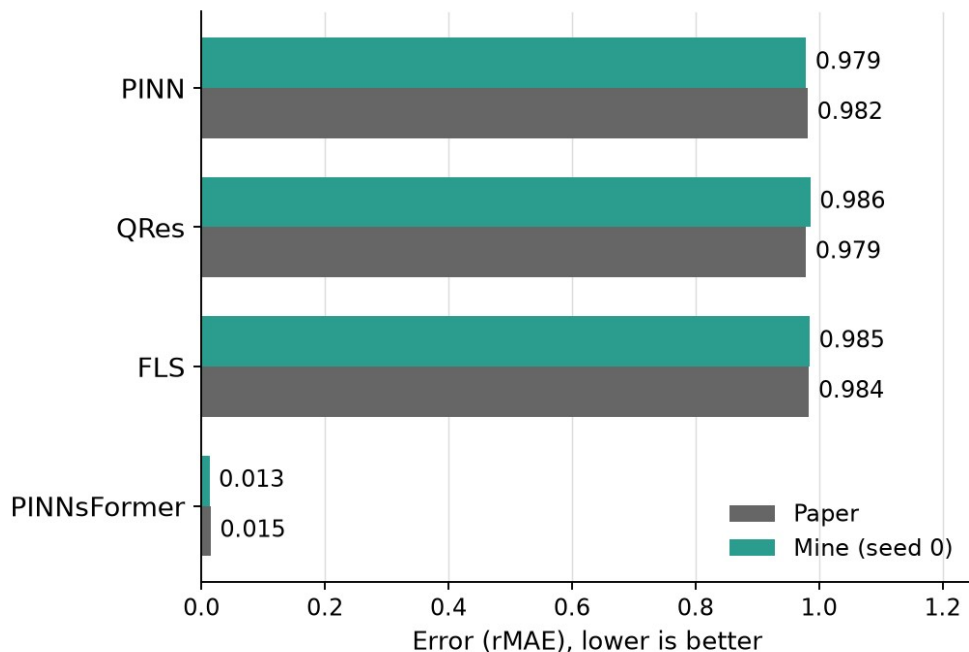
Things in the code that do not match the paper

Reading the code before running it, I found several differences. None is documented.

- Training length: the paper says 1000 iterations, the notebooks run 500.
- Training grid: the baselines get 101×101 points, PINNsFormer only 51×51 . The paper calls this data efficiency.
- Every result in the paper is a single run, with no variance.
- The wave equation coefficient is 3 in the text but 4 in the code (and in the paper's own solution).
- One baseline (QRes) is configured differently in two notebooks.
- A layer width (256) is not in the paper, and one figure caption names the wrong equation.

The first three matter for the results. The rest are small but show how hard it is to reproduce from the paper alone.

Result 1: on 1D-reaction, the paper reproduces



What the chart shows

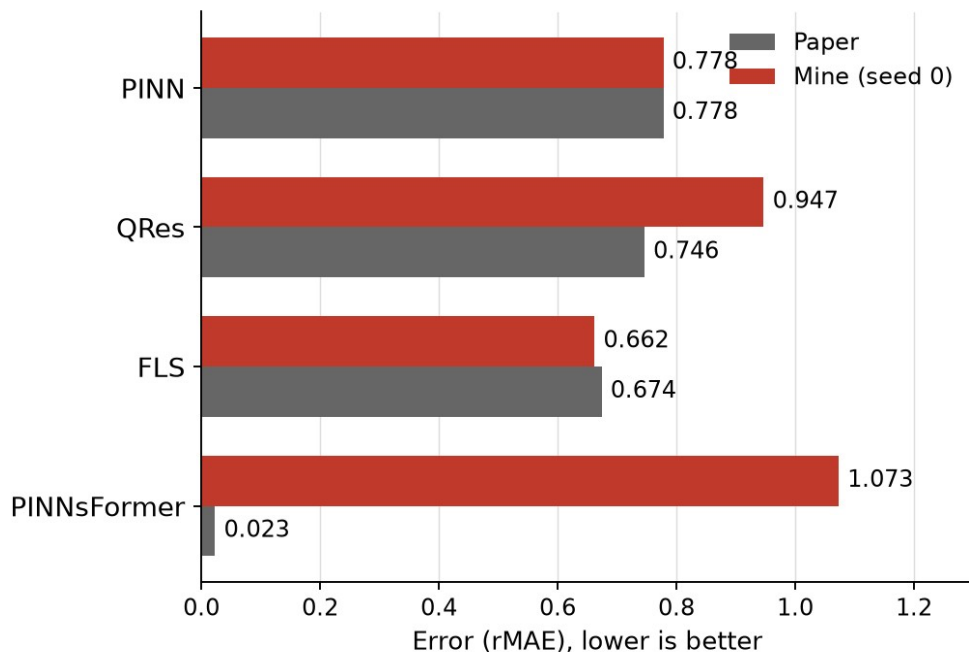
The three baselines have an error near 1 (the flat wrong solution). PINNsFormer is near 0. My numbers match the paper's.

Over 3 seeds: 0.016 ± 0.003 (paper: 0.015).

One surprise

The plain PINN solved the problem on one of my three seeds (error 0.06). So 'the PINN fails' is true most of the time, not always.

Result 2: on convection, it did not reproduce



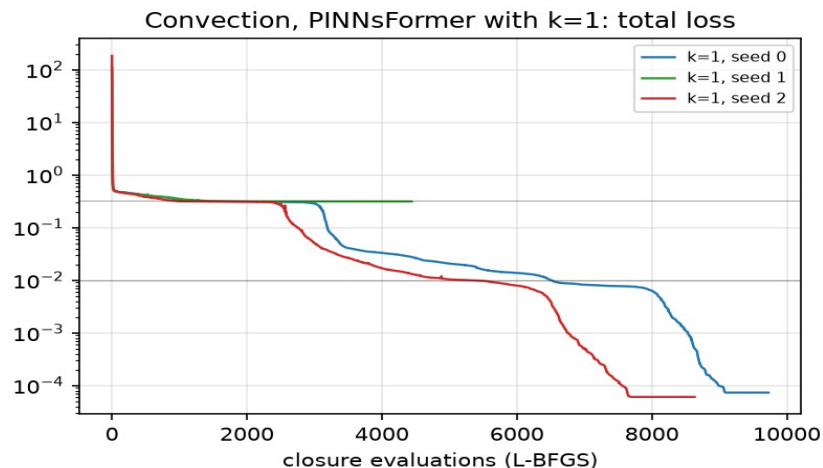
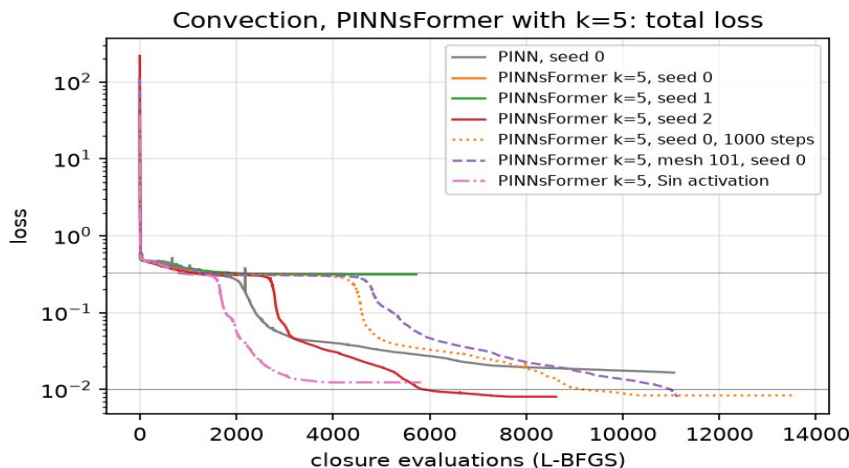
What the chart shows

The baselines match the paper. PINNsFormer does not: the paper reports 0.023, I got 0.54 to 1.07 depending on the seed.

Is the model broken? No.

- The authors' saved model (checkpoint), evaluated with my code, gives 0.023.
- Running 1000 steps instead of 500 did not help (0.55).
- So the model can represent the solution. Their training run reached it; mine did not.

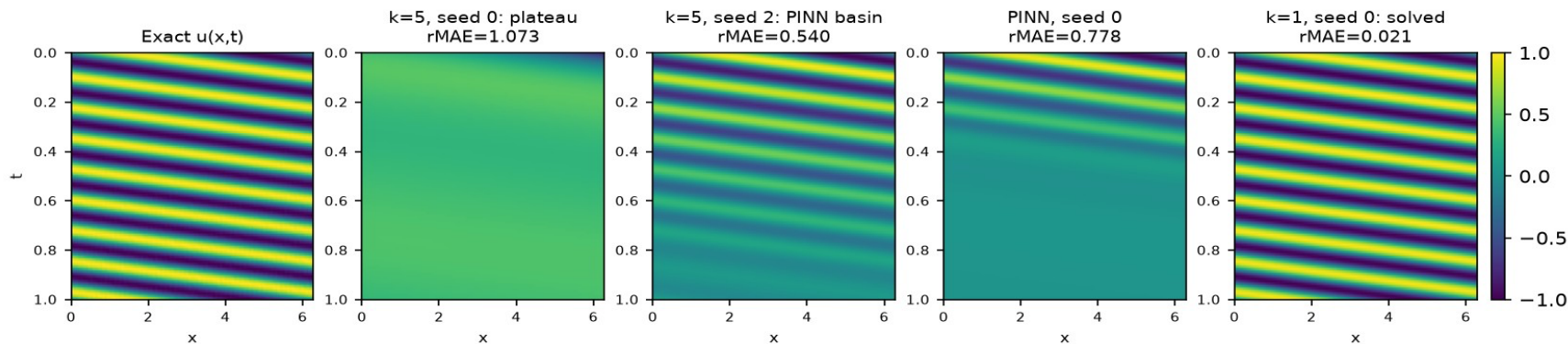
Why? Looking at the training curves



Loss (log scale) during training, every PINNsFormer run on convection. Left: the paper's setup. Right: my $k = 1$ runs.

- Every run first sits on a plateau (loss 0.33): the network outputs a constant and has not even fitted $t = 0$. Two seeds never leave it.
- Runs that escape fall into the plain-PINN failure (loss 0.01). Only some go on to the real solution (loss 0.0001).
- Which happens depends on the random seed. The authors' published run was one that made it.

My extra experiments: change one thing at a time



Experiment	What it tests	Convection	1D-reaction
Plain PINN + Wavelet activation	Is it just the activation?	fails (1.26)	fails (0.98)
PINNsFormer with $k = 1$ (no sequence)	Is it the sequence / attention?	works (0.021), 2 of 3 seeds	fails (0.98)
PINNsFormer on the big 101×101 grid	Would more points help?	worse (0.53)	fails (0.98)

So on convection the sequence is not what makes it work: without any sequence it works better. On reaction the sequence does matter.

Two more results

Is the Wavelet activation necessary?

- With ReLU: fails on both equations, as the paper says.
- With a plain sine: works as well as Wavelet on reaction and better on convection (0.70 vs 1.07).

Why? $\sin(x) + \cos(x) = \sqrt{2} \cdot \sin(x + \pi/4)$. Wavelet is a sine with a learnable amplitude and phase. It is not surprising it behaves like one.

Wave equation and cost

- On the wave equation my plain PINN (0.10) beat PINNsFormer (0.37) and was 3× better than the paper's own PINN (0.33).
- Cost: about 2× slower than a PINN with the paper's grids, but 11× slower and 9× more memory when both use the same 101×101 grid.

What I conclude

Claim 1: fixes the failure modes	Yes on 1D-reaction. On convection only via the authors' saved model, not by training.	Partly
Claim 2: Wavelet is necessary	Compared to ReLU yes, compared to a plain sine no.	Partly
Claim 3: works on the wave equation	My plain PINN did better.	No
Claim 4: small overhead	True on their grids, not on equal grids.	Partly
Claim 5: it works because of temporal dependencies	Without any sequence it worked better on convection.	No

In one sentence: the method works on one of the two equations it was built on; on the other, the authors' run was a lucky one, and their explanation for why it works does not survive the test without a sequence.

Limitations, and what I learned

Limitations of my study

- Only 3 seeds, 2 equations, 500 steps. Enough to see that results are unstable, not to measure how unstable.
- My GPU and PyTorch version differ from the authors', and L-BFGS is sensitive to tiny numerical differences.
- For the activation test I had to guess one detail the paper does not specify.

What I learned

- Read the code before running it: the paper and the code disagreed in eight places.
- Always run more than one seed. Both the baselines and the new method changed outcome with the seed.
- A single good number can come from a lucky run. The training curves tell you more than the final table.
- Test the explanation, not just the result: a control with no sequence was the most informative experiment I ran.